

HITO 5 · ENTREGA FINAL Y DEFENSA

POWER STRIKE

Sistema de gestión de gimnasio · Trabajo de Campo · Ingeniería de Software II

Grupo 2: Mateo Brizuela · Ernesto Ponce · Martín Mousist · Fabricio Posada

Defensa: martes 16/06/2026 · 19:00 · github.com/MartinMousist/power-strike

AGENDA

Cómo organizamos la defensa

Mateo

Plan SQA, umbrales de calidad y trazabilidad (RTM). Testing automatizado: caja negra y caja blanca. Diseño de casos de prueba. Métricas finales: cobertura, complejidad y mantenibilidad.

Martín

Arquitectura en capas y stack. Ejemplos de pruebas e integración continua (CI). TDD: ciclo Red-Green-Refactor. Gestión de defectos y cierre del Hito 5 con los defectos corregidos. Implementación de pruebas de caja blanca y caja negra

Fabrizio

Análisis de riesgos (FMEA) y usabilidad: heurísticas de Nielsen + wireframes en Figma.

Ernesto

El producto: qué resuelve Power Strike y sus módulos. Máquina de estados de la asistencia.

Qué resuelve Power Strike

Administrar usuarios, actividades y asistencias de un gimnasio, con autenticación por roles y una API REST consumida por una SPA.

Usuarios

Alta con nombre, email y DNI validados. Roles ADMIN / TRAINER / CLIENT.

Actividades

Crear, editar y consultar: nombre, día, horario y costo mensual.

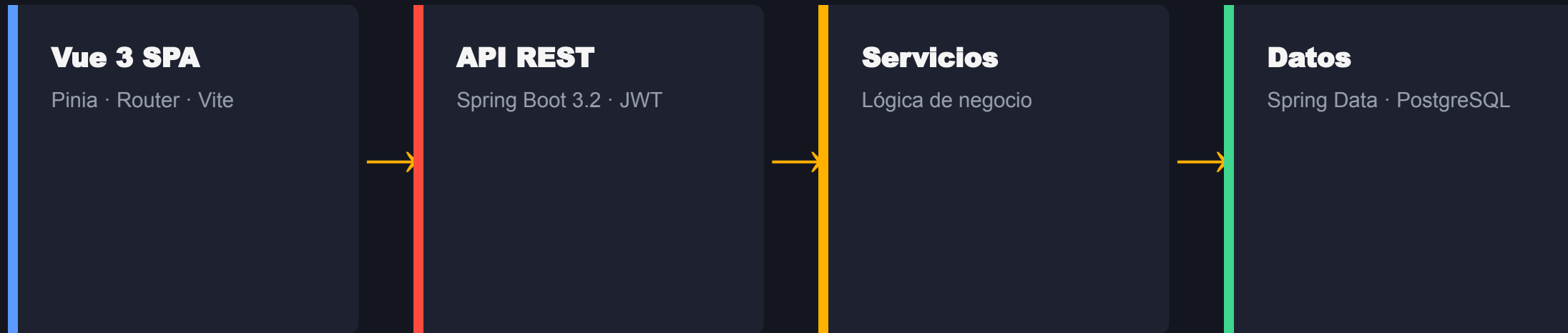
Asistencias

Registro e historial de asistencia por usuario.

Seguridad

Login con JWT y autorización por rol en los endpoints.

Capas y stack tecnológico



Arquitectura en capas (controller → service → repository): cada capa se testea de forma aislada.

Plan SQA

Calidad guiada por el modelo ISO/IEC 25010.

Qué controlamos

Trazabilidad requerimiento→código→test, cobertura, complejidad y mantenibilidad, y gestión de defectos como Issues.

Cómo medimos

En cada push: tests + cobertura + linters (CI). Por hito: revisión de métricas y actualización del plan.

Umbrales

Cobertura $\geq 60\%$ en módulos nuevos · CC ≤ 10 · 100% tests en verde · 0 críticos abiertos al mergear.

RTM - Requerimientos funcionales

REQ	Módulo / función	Test	Estado
REQ-F01 · Registrar usuarios	UserController.createUser	UserControllerTest (CP-01..20)	✓ OK
REQ-F02 · Listado de usuarios	UserController.getAllUsers	UserControllerTest	✓ OK
REQ-F03 · Gestionar actividades	ActivityController	ActivityControllerTest (AC-01..10)	✓ OK
REQ-F04 · Mostrar actividades	ActivityController.getAllActivities	ActivityControllerTest	✓ OK
REQ-F05 · Asistencias	AttendanceController	AttendanceControllerTest (AT-01..06)	✓ OK

RTM - No funcionales y seguridad

REQ-NF01 -JWT

Autenticación con token. AuthController + JwtUtil + filtro. Test de login.

REQ-NF02 -Roles

Autorización por rol con @PreAuthorize.
CLIENT→403, ADMIN→200. Verificado por test de seguridad. DEF-EXP-04 corregido (Hito 5).

REQ-NF03 -Errores

GlobalExceptionHandler: 400 / 409 / 404 / 403 coherentes. Cubierto por tests EX.

El control de roles (DEF-EXP-04) era el hueco crítico del Hito 4: se cerró en el Hito 5.

Testing automatizado

56 - caja negra

Controllers vía MockMvc

Validaciones y errores probados por HTTP, sin levantar la base. Verifican la especificación.

22 - caja blanca

Services con Mockito

Lógica de negocio con dobles de prueba. Cubren ramas y casos límite.

Técnicas: partición de equivalencia · valores límite · tabla de decisión · máquina de estados.

Ejemplos de pruebas

Prueba de caja negra (CP03)

```
@Test
void CP03_dni6Digitos_devuelve400() throws Exception {
    post400(VALID_NAME, VALID_EMAIL, "123456", VALID_PASSWORD, VALID_ROLE);
}
```

caja blanca

```
@Test
void updateUser_cuandoPasswordEsBlank_noReencodea() {
    User datosActualizados = new User(null, "Juan Actualizado", "juan@email.com", "12345678", "", "ADMIN", true);
    when(userRepository.findById(1L)).thenReturn(Optional.of(usuarioAdmin));
    when(userRepository.save(usuarioAdmin)).thenReturn(usuarioAdmin);

    userService.updateUser(1L, datosActualizados);

    verify(passwordEncoder, never()).encode(any());
    verify(userRepository, times(1)).save(usuarioAdmin);
}
```

TDD - ciclo Red-Green-Refactor

1 - RED

Escribimos primero el test del caso de uso. Falla, porque la funcionalidad o la validación todavía no existe.

2 - GREEN

Implementamos lo mínimo para que el test pase. La barra se pone verde.

3 - REFACTOR

Limpiamos el código con la red de seguridad de los tests ya en verde.

El ciclo queda visible en el historial de commits: primero el test que falla, después el fix.

Máquina de estados - Asistencia



Modelamos la asistencia como estados y derivamos casos: primer registro OK, segundo registro del día → 409.

Integración continua

En cada push

GitHub Actions compila, corre los 78 tests y genera el reporte de cobertura JaCoCo.

Feedback inmediato

Si un test falla, el pipeline avisa antes de mergear. La rama main siempre queda verde.

Análisis estático

Checkstyle y PMD sobre el código Java; ESLint en el frontend.

La calidad deja de depender de la disciplina individual: la verifica la máquina en cada cambio.

Tamaño y complejidad

1.585

líneas de producción

Backend 704 + Frontend 881. Tests: 839 (razón 0,53).

CC $\leq$ 10

complejidad ciclomática

0 violaciones en PMD. Método más complejo: CC = 3.

MI $\approx$ 85

mantenibilidad (MI)

Escala SEI: ≥ 85 es alta mantenibilidad.

Cobertura y resultado

100%

**cobertura en controller,
service y model**

Supera el umbral de 60% del Plan SQA.

53%

cobertura global

El resto es security/ y arranque, fuera del testing funcional.

78

tests - todos en verde

0 fallos, 0 errores en la suite completa.

Evolución: 21 tests (H3) → 72 (H4) → 78 (H5). Cobertura de módulos principales: parcial → 100%.

12 detectados - 9 cerrados

6

documentados

TDD y análisis

6

exploratorios

sesiones manuales

9

corregidos / cerrados

incl. el crítico de roles

3

abiertos

próximos avances

Severidad (QA) y Prioridad (PO) separadas · todos cargados como Issues en GitHub.

Defectos corregidos con TDD

DEF-EXP-04

Sin control de roles (CLIENT accedía a todo)

→ **403 con @PreAuthorize**

DEF-EXP-02

Asistencias duplicadas el mismo día

→ **409 Conflict**

DEF-EXP-03

No se podía editar usuario sin cambiar pass

→ **200 (grupo OnCreate)**

DEF-EXP-01

userId inexistente devolvía 500

→ **404 NotFound**

DEF-EXP-06

Día de actividad sin validar

→ **400 con @Pattern**

FMEA

Modos de falla

Identificamos qué puede fallar en el registro de usuarios y su efecto en el sistema.

NPR

Priorizamos por Número de Prioridad de Riesgo
= gravedad × ocurrencia × detección.

Mitigaciones

Los riesgos altos se tradujeron en validaciones y tests concretos del backend.

Modo de fallo mas critico como ejemplo:

#	Modo de fallo	Efecto	S	P	D	RPN
1	Falla de base de datos durante el guardado de la asistencia	La asistencia no se registra y se pierde información importante sobre la concurrencia de los usuarios	8	5	8	320

FMEA

Mitigación Propuesta

- **Modo de fallo:** Falla de base de datos durante el guardado (RPN 320).
- **Estrategia:** Detección + Recuperación.
- **Implementación concreta:** Registrar errores mediante logs y devolver un mensaje controlado al usuario. Implementar reintentos automáticos de conexión y monitoreo de disponibilidad de la base de datos para detectar fallas tempranamente.

Análisis heurístico - 10 de Nielsen

Evaluamos las 5 pantallas con las 10 heurísticas de Nielsen. Hallazgos por severidad:

2

mayores

confirmar al borrar · mensaje al borrar con asistencias · buscar usuario

5

menores

loading · validación front · filtros/paginación · ayuda contextual

3

cosméticos

mensajes técnicos · consistencia de botones · dashboard vacío

Top 3 mejoras: confirmación al eliminar · mensaje correcto al borrar (FK) · selector de usuario en asistencias.

Prototipos figma

Pantalla: dashboard

Dashboard

Actividades

Salir

Gestión de Actividades

5 actividades registradas

+ Nueva Actividad

Nombre	Día	Horario	Cuota	Instructor	Acciones
					Editar Eliminar
					Editar Eliminar
					Editar Eliminar
					Editar Eliminar
					Editar Eliminar

Prototipos figma

Pantalla: dashboard (alta fidelidad)

 **Power Strike**

Dashboard

Usuarios

Actividades

Asistencia

AdministradorADMIN

Salir

Dashboard

Bienvenido, **Administrador**



TOTAL USUARIOS

248



ACTIVIDADES

14



ASISTENCIAS HOY

73



TOTAL ASISTENCIAS

1.840

Últimas Asistencias

Usuario	Fecha y Hora	Notas
Lucía Fernández	12/06/2026 18:42	Musculación
Martín Gómez	12/06/2026 18:35	Crossfit
Valentina Torres	12/06/2026 18:20	Funcional
Diego Ramírez	12/06/2026 18:10	Yoga
Camila López	12/06/2026 17:58	Musculación



Prototipos figma

Pantalla: gestión de actividades

Dashboard

Actividades

Salir

Gestión de Actividades

5 actividades registradas

+ Nueva Actividad

Nombre	Día	Horario	Cuota	Instructor	Acciones
					Editar Eliminar
					Editar Eliminar
					Editar Eliminar
					Editar Eliminar
					Editar Eliminar

Prototipos figma

Pantalla: gestión de actividades (alta fidelidad)

 Power Strike

Dashboard

Usuarios

Actividades

Asistencia

AdministradorADMIN

Salir

Gestión de Actividades

+ Nueva Actividad

5 actividades registradas

Nombre	Día	Horario	Cuota	Instructor	Acciones
Musculación	Lunes	18:00	\$25.000	Carlos Pérez	EditarEliminar
Crossfit	Martes	20:00	\$30.000	Sofía Ruiz	EditarEliminar
Funcional	Miércoles	19:00	\$22.000	Lucas Méndez	EditarEliminar
Yoga	Jueves	09:00	\$18.000	Ana Gómez	EditarEliminar
Natación	Viernes	07:30	\$35.000	Martín Torres	EditarEliminar

Demo y wireframes

▶ DEMO EN VIVO

Login → Usuarios → Actividades → Asistencias → intento como CLIENT (403)

Wireframes y mockups de las pantallas clave: proyecto de Figma del equipo ([link](#)).

Qué aprendimos - qué haríamos distinto

Aprendimos

Testing como parte del desarrollo (TDD) · técnicas formales de diseño de casos · métricas que importan (cobertura útil, CC, MI) · integración continua · trazabilidad.

Haríamos distinto

Testear desde el día 1 · implementar RBAC más temprano · validar también en el frontend · cubrir seguridad y frontend con tests · cerrar antes los defectos abiertos.

CONCLUSIÓN

Un producto funcional y un método de calidad replicable

78 tests verdes · 100% en módulos principales · CI activo · trazabilidad completa · defecto crítico cerrado

¡Gracias!